Introduction to Modules and Procedures and function

Introduction

A module is a file, also called a source file (as in C/C++, Pascal, and other languages) that belongs to a program and in which you write code. For example, while we were using forms so far, we couldn't write code on a form. We had to access the Code Editor.

Practical Learning: Introducing Modules

1. Start Visual Basic and, in the opening dialog box, double-click Standard EXE
2. To save the project, on the main menu, click File -> Save Project
3. Locate your MS Visual Basic folder created in the first lesson and display it in the Save In combo box
4. In the Save File As dialog box, click the Create New Folder button
5. Type **Modules1** and press Enter twice to display the new folder in the Save In combo box
6. Replace the name of the file in the File Name edit box with **Main** and press Enter
7. Type **Modules1** to replace the name of the project and press Enter. If are asked whether you want to add the project to SourceSafe, click No
8. Display the form and design it as follows:

Control	Name	Caption	Additional Properties
Label		Side:	
TextBox	txtSide	0.00	Alignment: 1-Right Justify
Label		Perimeter:	
TextBox	txtPerimeter	0.00	Alignment: 1-Right Justify
Button	cmdCalcPerimeter	Calculate	
Label		Area:	
TextBox	txtArea	0.00	Alignment: 1-Right Justify
Button	cmdCalcArea	Calculate	

9. Save all

Creating a Module

There are two main ways you obtain a module for your program.

- When you create a form, a module is automatically created and associated to it.

- You can access the module of a form by right-clicking it and click View Code.
- Besides the modules created for your forms, you can create your own modules any time.
- To create a new module, on the main menu, you can click **Project -> Add Module**.
- In the Add Module dialog box, you can click Open.
- Once a new module has been added to your project, it is treated as a file.
- As such, you should save it to give it a friendlier name.
- The name of a module can be used to indicate the type of code included in that module.

Once a module is ready,

- you can type any piece of valid code in it.
- Such a module is automatically made available to other parts of your program.
- Just as you can write code in a module attached to a form, you can also write code in an independent module (unlike some other languages like C/C++ or Pascal, you usually don't need to reference one module from another; once a module has been created, you can use its contents as you wich).

Any existing module of your project is represented in the Project window with a button and a name in the Modules node. Therefore, to open a module, in Project window, you can double-click its node.

Practical Learning: Creating a Module

1. To access the form's module, double-click anywhere in the form and notice that a source file opens
2. To create a new module, on the Standard toolbar, click the arrow of the Add Form button and click Module
3. In the Add Module dialog box, make sure the Module icon is selected and click Open
4. To save the new module, on the Standard toolbar, click the Save button
5. Change the name of the file to **modGeometry** and press Enter
6. To change the name of the module in Visual Basic, in the Project window, click Module1. In the Properties window, click (Name). Type **Routines** and press Enter

We use procedures and functions to create modular programs. Visual Basic statements are grouped in a block enclosed by `Sub`, `Function` and matching `End` statements. The difference between the two is that functions return values, procedures do not.
A procedure and function is a piece of code in a larger program. They perform a specific task. The advantages of using procedures and functions are:

- Reducing duplication of code
- Decomposing complex problems into simpler pieces

- Improving clarity of the code
- Reuse of code
- Information hiding

Procedures

A procedure is a block of Visual Basic statements inside `Sub`, `End Sub` statements. Procedures do not return values.

```
Option Strict On


Module Example

    Sub Main()

        SimpleProcedure()

    End Sub

    Sub SimpleProcedure()
        Console.WriteLine("Simple procedure")
    End Sub

End Module
```

This example shows basic usage of procedures. In our program, we have two procedures. The `Main()` procedure and the user defined `SimpleProcedure()`. As we already know, the `Main()` procedure is the entry point of a Visual Basic program.

```
SimpleProcedure()
```

Each procedure has a name. Inside the `Main()` procedure, we *call* our user defined `SimpleProcedure()` procedure.

```
Sub SimpleProcedure()
    Console.WriteLine("Simple procedure")
End Sub
```

Procedures are defined outside the `Main()` procedure. Procedure name follows the `Sub` statement. When we call a procedure inside the Visual Basic program, the control is given to that procedure. Statements inside the block of the procedure are executed. Procedures can take optional parameters.

```
Option Strict On


Module Example
```

```
    Sub Main()

        Dim x As Integer = 55
        Dim y As Integer = 32

        Addition(x, y)

    End Sub

    Sub Addition(ByVal k As Integer, ByVal l As Integer)
        Console.WriteLine(k+l)
    End Sub

End Module
```

In the above example, we pass some values to the `Addition()` procedure.

```
Addition(x, y)
```

Here we call the `Addition()` procedure and pass two parameters to it. These parameters are two Integer values.

```
Sub Addition(ByVal k As Integer, ByVal l As Integer)
    Console.WriteLine(k+l)
End Sub
```

We define a *procedure signature*. A procedure signature is a way of describing the parameters and parameter types with which a legal call to the function can be made. It contains the name of the procedure, its parameters and their type, and in case of functions also the return value. The `ByVal`keyword specifies how we pass the values to the procedure. In our case, the procedure obtains two numerical values, 55 and 32. These numbers are added and the result is printed to the console.

Functions

VB 6 Declaring Functions

A Vb 6 subroutine receives an argument but does not return anything. However, a function takes an argument and returns a value. A VB 6 function is a block of code that can separate your program into manageable parts.

Declaring a VB 6 Function

```
[Private | Public | Friend] [Static] Function name [(arglist)] [As type]

...

[Statements]

...

[expression]

...

[Exit Function]

...

[Statement]

...

End Function
```

- **Public** keyword – All procedures in all other modules and forms can access the function.
- **Private** keyword – All procedures and function in the same module or form where the function is declared can access it.
- **Friend** keyword – Used only in modules that uses classes and it means that all procedures throughout the project can access this function except the controller of an instance of an object.
- **Static** – Normally, a local variable to a function get destroyed when function terminates. Static keyword keeps the value of a local variable intact between function calls.
- ***Function name*** – User-defined name for the function.
- ***arglist*** – A list of variables passed as arguments to the function.
- ***type*** – The return data type of the function.
- **Statements** – A group of statements that a function can execute.
- **Exit Function** – A way to exit the function immediately before it could complete execution.
- **End Function** – Terminates the function upon completion.

How to declare arglist in the function?

Arguments to a function are values that the function will use to complete a calculation or complete a task.

To call a function you have to use the function name and the list of arguments which uses the following syntax.

```
[Optional][ByVal|ByRef][ParamArray] varname[()] [As type][= defaultvalue]
```

Now, we will briefly discuss the list above.

`Optional` – This keyword means that the arglist is optional and not required.

`ByRef` – This keyword means that a reference to the original value is passed through the function.This is true by default.

`ByVal` – This keyword means that the original value of the variable is passed to the function.

`ParamArray` – Usually you can send limited number of variables through a function, however, if you need to send a list of values of same type use the ParamArray or nothing at all, because this is optional.

`varname` – It is the programmer-defined name of the argument.

`type` – This defines the data type of the argument.

`defaultvalue` – If you want to set a optional default value for a argument, this feature is helpful. If no value is supplied for a variable, then the function can use the default value for computation.

Example Program: VB Function

In this example, we will create a simple function that accepts two numbers and return their `sum`.

VB Code

```
Private Sub Command1_Click ()

Dim result As Integer

result = Sum (5, 23)

MsgBox ("Result = " + Str(result))

End Sub

Function Sum (number1 As Integer, number2 As Integer) As Integer

Sum = number1 + number2

End Function
```

Output

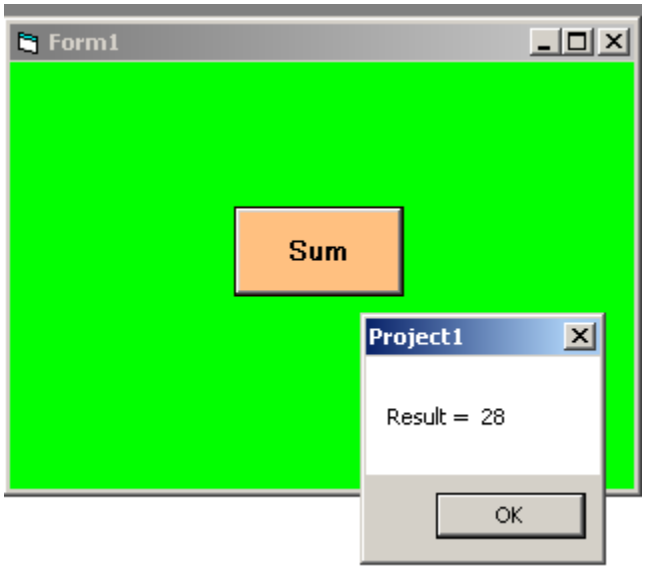

Figure 1 – Output- VB Function Add

A function is a block of Visual Basic statements inside `Function`, `End Function` statements. Functions return values.

There are two basic types of functions. **Built-in functions and user defined Functions**. The built-in functions are part of the Visual Basic language. There are various mathematical, string or conversion functions.

```
Option Strict On

Module Example

    Sub Main()

        Console.WriteLine(Math.Abs(-23))
        Console.WriteLine(Math.Round(34.56))
        Console.WriteLine("ZetCode has {0} characters", _
            Len("ZetCode"))

    End Sub

End Module
```

In the preceding example, we use two math functions and one string function. Built-in functionshelp programmers do some common tasks.

In the following example, we have a user defined function.

```
Option Strict On


Module Example

    Dim x As Integer = 55
    Dim y As Integer = 32

    Dim result As Integer

    Sub Main()

        result = Addition(x, y)
        Console.WriteLine(Addition(x, y))

    End Sub

    Function Addition(ByVal k As Integer, _
                      ByVal l As Integer) As Integer
        Return k+l
    End Function

End Module
```

Two values are passed to the function. We add these two values and return the result to the `Main()` function.

```
result = Addition(x, y)
```

Addition function is called. The function returns a result and this result is assigned to the result variable.

```
Function Addition(ByVal k As Integer, _
                  ByVal l As Integer) As Integer
    Return k+l
End Function
```

This is the Addition function signature and its body. It also includes a return data type, for the returned value. In our case is is an `Integer`. Values are returned to the caller with the `Return`keyword.

How to pass a function for parameter using VB6?

Solution 1:

As far as I remember, VB6 has a function called **`AddressOf`** which allows you to obtain function addresses. However, it may be quite challenging to utilize that function address from within VB6.

The standard operating procedure for managing this involves utilizing **`CallByName()`** , which enables the retrieval of function names and other related features through calls.

To tackle this issue, an alternative solution is to implement the standard OO approach. Instead of directly passing the function, you can create a class that adheres to a customized interface. The interface should include a single method that can be implemented in various classes to access different functions. Afterwards, you can provide an instance of one of these classes to your function, which will receive it as an argument and invoke the designated method. However, this approach requires several minor design modifications.

Solution 2:

Passing a function is not possible, but an object that can act as a function (known as a "functor") can be passed. This is a common technique I employ frequently. When an interface is implemented by the "functor" class, the call becomes type-safe. As an illustration:

The interface IAction.cls is an abstract class.

```
Option Explicit
```

```
Public Sub Create(ByVal vInitArgs As Variant)

End Sub

Public Function exe() As Variant

End Function
```

A functional object that opens a URL in the default web browser.

```
Option Explicit

Implements IAction

Dim m_sUrl As String

Public Sub IAction_Create(ByVal vInitArgs As Variant)

m_sUrl = vInitArgs

End Sub

Public Function IAction_exe() As Variant

    Call RunUrl(m_sUrl) 'this function is defined elsewhere

Exit Function
```

It is now possible to generate multiple instances of these classes, compile them into a group, transfer them to any function or method that requires an IAction, and so on.

Returning Arrays from Functions

Example#

A function in a normal module (but not a Class module) can return an array by putting `()` after the data type.

```
Function arrayOfPiDigits() As Long()
    Dim outputArray(0 To 2) As Long

    outputArray(0) = 3
    outputArray(1) = 1
    outputArray(2) = 4

    arrayOfPiDigits = outputArray
End Function
```

The result of the function can then be put into a dynamic array of the same type or a variant. The elements can also be accessed directly by using a second set of brackets, however this will call the function each time, so its best to store the results in a new array if you plan to use them more than once

```
Sub arrayExample()

    Dim destination() As Long
    Dim var As Variant

    destination = arrayOfPiDigits()
    var = arrayOfPiDigits

    Debug.Print destination(0)          ' outputs 3
    Debug.Print var(1)                  ' outputs 1
    Debug.Print arrayOfPiDigits()(2)    ' outputs 4

End Sub
```

Note that what is returned is actually a copy of the array inside the function, not a reference. So if the function returns the contents of a Static array its data can't be changed by the calling procedure.

Outputting an Array via an output argument#

It is normally good coding practice for a procedure's arguments to be inputs and to output via the return value. However, the limitations of VBA sometimes make it necessary for a procedure to output data via a `ByRef` argument.

Outputting to a fixed array#

```
Sub threePiDigits(ByRef destination() As Long)
```

```
    destination(0) = 3
    destination(1) = 1
    destination(2) = 4
End Sub

Sub printPiDigits()
    Dim digits(0 To 2) As Long

    threePiDigits digits
    Debug.Print digits(0); digits(1); digits(2) ' outputs 3 1 4
End Sub
```

Outputting an Array from a Class method#

An output argument can also be used to output an array from a method/proceedure in a Class module

```
' Class Module 'MathConstants'
Sub threePiDigits(ByRef destination() As Long)
    ReDim destination(0 To 2)

    destination(0) = 3
    destination(1) = 1
    destination(2) = 4
End Sub

' Standard Code Module
Sub printPiDigits()
    Dim digits() As Long
    Dim mathConsts As New MathConstants

    mathConsts.threePiDigits digits
    Debug.Print digits(0); digits(1); digits(2) ' outputs 3 1 4
End Sub
```

Visual Basic Optional Parameters

In visual basic, the **optional parameters** have been introduced to specify the parameters as optional while defining the methods, indexers, constructors, and delegates.

Generally, while calling the method we need to pass all the method parameters. In case, if we specify a few of method parameters as an optional, then we can omit those parameters while calling the method because the optional parameters are not mandatory.

If we want to make the parameter as an optional, we need to define a parameter with `Optional` keyword and assign the default value to that parameter as part of its definition in methods, indexers, constructors, and delegates.

While calling the method, if no argument is sent for an optional parameter, then the default value will be used otherwise it will use the value which is sent for that parameter.

Following is the example of defining the optional parameters in a method.

```
Public Sub GetDetails(ByVal id As Integer, ByVal name As String, Optional
ByVal location As String = "Hyderabad")
' your code
End Sub
```

If you observe the above method, we made the **location** parameter as optional by assigning the default value (**Hyderabad**). So, while accessing **GetDetails()** method, if no argument sent for **location** parameter, then by default it will consider "**Hyderabad**" value otherwise it will use the value which is sent for that parameter.

The defined **GetDetails()** method can be accessed as shown below.

```
GetDetails(1,"Suresh");
GetDetails(1,"Suresh", "Chennai");
```

While defining the optional parameters, we need to make sure that the optional parameters are defined at the end of the parameter list, after any required parameters like as we defined in the above method otherwise we will get the compile-time error like "*optional parameters must appear after all required parameters*".

Following is the **invalid** way of defining the optional parameters in a method.

```
Public Sub GetDetails(ByVal id As Integer, ByVal Optional location As Stri
ng = "Hyderabad", ByVal name As String)
' your code
End Sub
```

The above method will throw the compile-time error because we defined a required parameter (**name**) after the optional parameter (**location**). As per rules, the optional parameters must be defined at the end of the parameter list.

Returning User defined types from Functions in VB6

order to return from a function, you do something as follows –

```
Private Function Add(ByVal x As Integer, ByVal y As Integer) As Integer

   Dim Res as integer

   Res = x + y

   Add = Res       ' use the function's name
```

```
End Function
```

My question is, does this syntax also work for user defined types? If not, what is the syntax. I tried the following –

```
Public Function getDetails() As clsDetails



Dim details As clsDetails



Set details = New clsDetails



With details

   .X = "R"

   .Y = "N"

   .Z = "N"

   ' more code follows

End With



getDetails = details 'gives error-> object variable or with block variable not set



End Function
```

But this gives me an error on the above line – "object variable or with block variable not set".

Functions returning arrays

Actually, you can return an array (or any type for that matter) by specifying the return type as a **Variant**.

Try this function:
VB Code:

```
1. Function ReturnsArray(a As Integer, b As Integer, c As Integer)
   As Variant
2.
3.     Dim vList(2) As Integer
4.
5.     vList(0) = a
6.
7.     vList(1) = b
8.
9.     vList(2) = c
10.
11.    ReturnsArray = vList
12.
13.End Function
```

And you would call it like:
VB Code:

```
1. Dim vArr As Variant
2.
3. vArr = ReturnsArray(1, 3, 5)
4.
5.
6. For I = 0 To UBound(vArr)
7.
8.     Debug.Print vArr(I)
9.
10.Next I
```

Creating a Visual Basic Application Containing Multiple Forms

Before we can look at hiding and showing Forms we first need to create an application in Visual Studio which contains more than one form. Begin by starting Visual Studio and creating a new Windows Application project called vbShowForm (see Creating a New Visual Basic Project for details).

Visual Studio will prime the new project with a single form. Click on the Form to select it and, using the *Properties* panel change the *Name* of the form to *mainForm*. Next we need to add a second form. To do this, click on the *Add New Item* in the Visual Studio toolbar to invoke the *Add New Item* window:

The *Add New Item* window allows new items of various types to be added to the project. For this example we just need to add a new Windows Form to our application. With *Windows Form* selected in the window click on *Add*. Visual Studio will now display an additional tab containing the second form:

Switch between the two forms by clicking on the respective tabs. With Form2 visible click on the form and change the name of the form in the *Properties* panel to *subForm*.

Understanding Modal and Non-modal Forms

A Windows form can be displayed in one of two modes, modal and non-modal. When a form is non-modal it means that other forms in the other forms in the application remain accessible to the user (in that they can still click on controls or use the keyboard in other forms).

When a form is modal, as soon as it is displayed all other forms in the application are disabled until the modal dialog is dismissed by the user. Modal forms are typically used when the user is required to complete a task before proceeding to another part of the application. In the following sections we will cover the creation of both modal and non-modal forms in Visual Basic.

Writing Visual Basic Code to Display a Non-Modal Form

We are going to use the button control on *mainForm* to display *subForm* when it is clicked by the user. To do this, double click on the button control to display

the *Click* event procedure. In this event procedure we want to call the *Show()* method of the *subForm* to make it display. To achieve this, modify the *Click()* event handler as follows:

```
Private Sub Button1_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs)
          Handles Button1.Click

        subForm.Show()

End Sub
```

To test this code press **F5** to compile and run the application. When it appears click on the button in the main form and the sub form will appear. You will notice, since this is a non-modal form, that you can still interact with the main form while the sub-form is visible (i.e. you can still click on the button in the main form).

Writing Visual Basic Code to Display a Modal Form

We will now modify the event procedure for the button to create a modal form. To do so we need to call the *ShowDialog()* method of the *subForm*. Modify the *Click* event procedure of the *mainForm* button as follows:

```
Private Sub Button1_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs)
                Handles Button1.Click

        subForm.ShowDialog()

End Sub
```

Press **F5** once again to build and run the application. After pressing the button in the main form to display the sub form you will find that the main form is inactive as long as the sub form is displayed. Until the sub form is dismissed this will remain the case.

Close the running application.

Hiding Forms in Visual Basic

There are two ways to make a form disappear from the screen. One way is to *Hide* the form and the other is to *Close* the form. When a form is hidden, the form and all its properties and settings still exist in memory. In other words, the form still exists, it is just not visible. When a form is closed, the form is physically deleted from memory and will need to be completely recreated before it can be displayed again.

To hide a form it is necessary to call the *Hide()* method of the form to be hidden. Using our example, we will wire up the button on the *subForm* to close the form. Click on the tab for the second form (the *subForm*) in your design and double click on the button control to display the *Click* event procedure.

One very important point to note here is that the button control is going to hide its own Form. In this case, the event procedure must reference *Me* instead of referencing *subForm* by name. With this in mind, modify the procedure as follows:

```
Private Sub Button1_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs)
               Handles Button1.Click

        Me.Hide()

End Sub
```

Press **F5** to build and run the application. Click on the button in the main form to display the sub form. Now, when you press the button in the sub form, the form will be hidden.

Closing Forms in Visual Basic

As mentioned in the previous section, in order to remove a form both from the display, and from memory, it is necessary to *Close* rather than *Hide* it. In Visual Studio double click, once again, on the button in *subForm* to view the *Click* event procedure. Once again, because the button is in the form we are closing we need to use *Me* instead of *subForm* when calling the *Close()* method:

```
Private Sub Button1_Click(ByVal sender As System.Object,
ByVal e As System.EventArgs)
             Handles Button1.Click

        Me.Close()

End Sub
```

When the *subForm* button is pressed the form will be closed and removed from memory.
